CSE360 – COMPUTER ARCHITECTURE



Traffic Control System with Adaptive Control for Vehicles and Pedestrians

Smart Solutions for Vehicle Flow and Pedestrian Safety



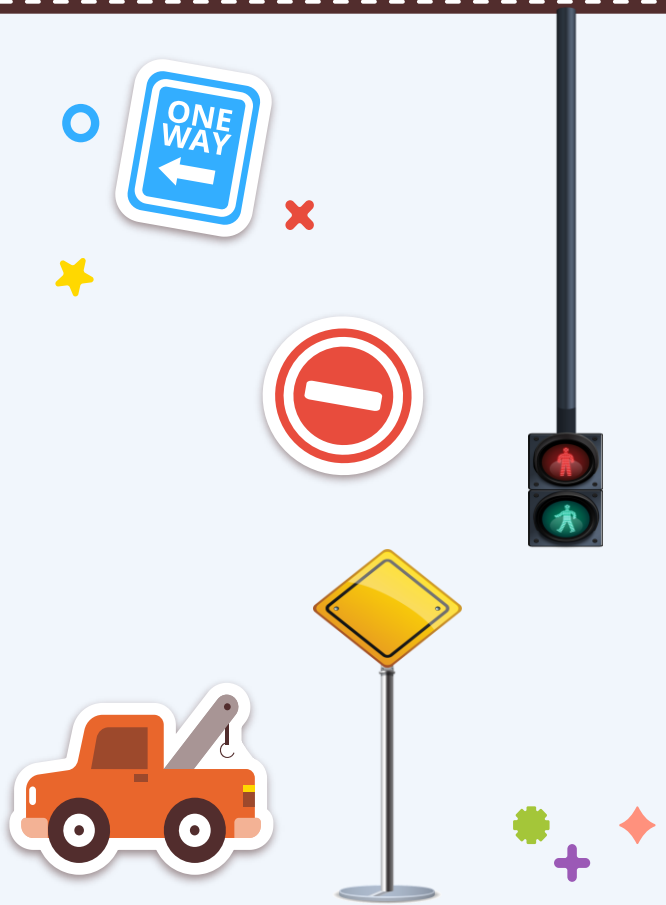


Presentation Details

Presented to: Nawab Yousuf Ali

Presented by:
Group Number: 7

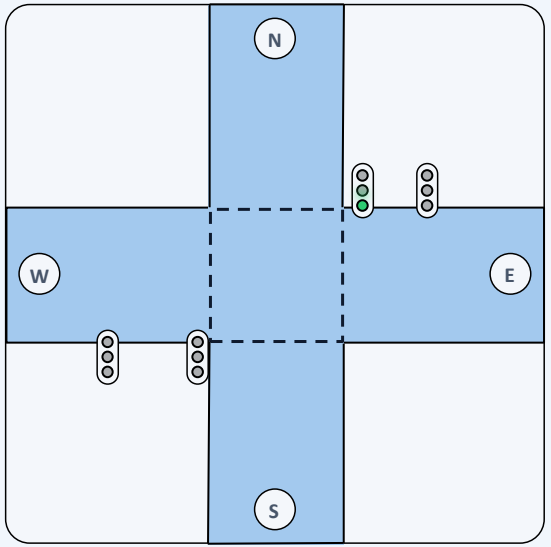
Members:	**ID:**
Nurtaz Reza	2023-3-60-282
Fabiha Shajjad Synthia	2023-3-60-398
Tanvir Tahsin	2023-3-60-385
Mohtasim Hauque	2023-3-60-439



Primary Objective

Simulate a 4-way traffic intersection with intelligent signal control

- Implement adaptive green time based on real-time vehicle density
- Handle emergency vehicles with priority override and Multiple directions
- Manage pedestrian crossing requests safely
- Process all 4 directions based on traffic priority
- Display real-time statistics per cycle and cumulatively



4-way intersection · North approach active (GREEN phase)

Traffic Signal Basics and Adaptive Concepts

1. Fixed-time vs. Adaptive Signals

Fixed-time signals cycle through predetermined green/yellow/red durations regardless of actual traffic. Adaptive signals read real-time vehicle density and vary signal duration dynamically — reducing unnecessary wait times on light-traffic approaches and extending green on congested ones.

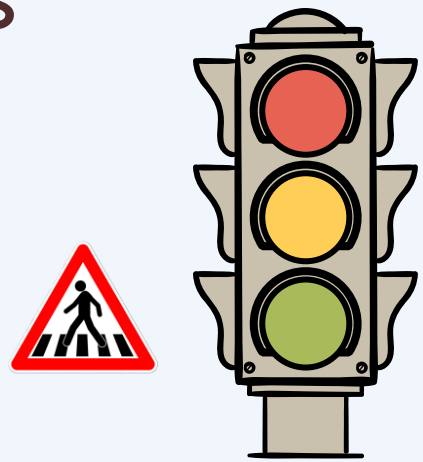
2. Signal Phase Sequence

Every direction follows the strict sequence GREEN → YELLOW → RED. GREEN permits vehicles to proceed, YELLOW warns of an imminent stop, and RED halts traffic until the next cycle grants a green. The phase order is enforced for all 4 roads, every cycle, without exception.

Traffic Signal Basics and Adaptive Concepts

3. Adaptive Control Principle

Green-light duration is computed per road from the live vehicle count on that approach. Higher density earns longer green time, capped at 15 seconds for safety. This principle mirrors real-world actuated signal controllers used in modern urban traffic management.



SIGNAL PHASES

GREEN 5–15s	YELLOW 3s
RED Until turn	PEDESTRIAN 7s

Adaptive Timing Formulas and Priority Scheduling

NORMAL TRAFFIC

Vehicle Count	Traffic Level	Green Time
0 – 5	Light	5 sec
6 – 15	Medium	10 sec
16+	Heavy	15 sec

EMERGENCY VEHICLE

Vehicle Count	Traffic Level	Green Time
1	Standard	8 sec
2 – 3	High	12 sec
4+	Critical	15 sec

PRIORITY SCHEDULING

Roads are sorted by vehicle density using qsort() — O(n log n) — with a custom descending comparator. The road with the most vehicles receives GREEN first. All 4 roads are processed every cycle, ensuring fairness. Inspired by priority scheduling from Operating Systems concepts.

System Architecture

MAIN CONTROL LOOP

STEP 01 **MAIN CONTROL LOOP** — main entry point, orchestrates every cycle

▼

STEP 02 **INPUT MODULE** — read vehicle counts for N/S/E/W directions

▼

STEP 03 **INPUT VALIDATION** — range-check, type-check, reject invalid values

▼

STEP 04 **EMERGENCY CHECK** — priority override if emergency vehicles are present

▼

STEP 05 **PRIORITY-BASED SCHEDULER** — sort roads by density using `qsort()`

▼

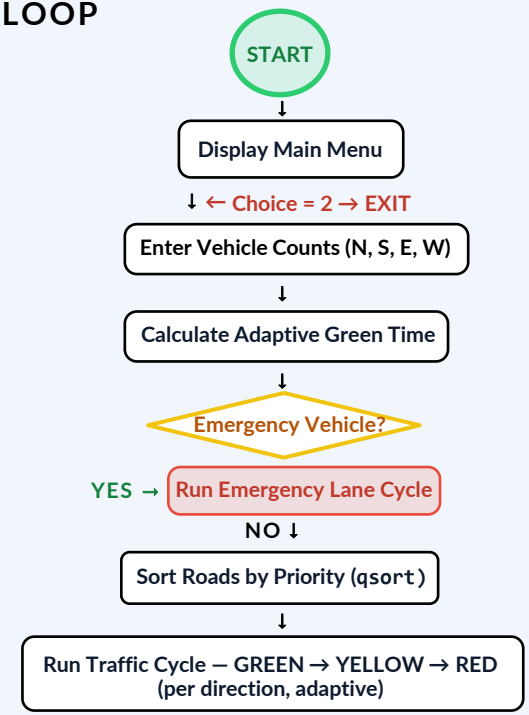
STEP 06 **SIGNAL CYCLE** — GREEN → YELLOW → RED per direction, adaptive timing

▼

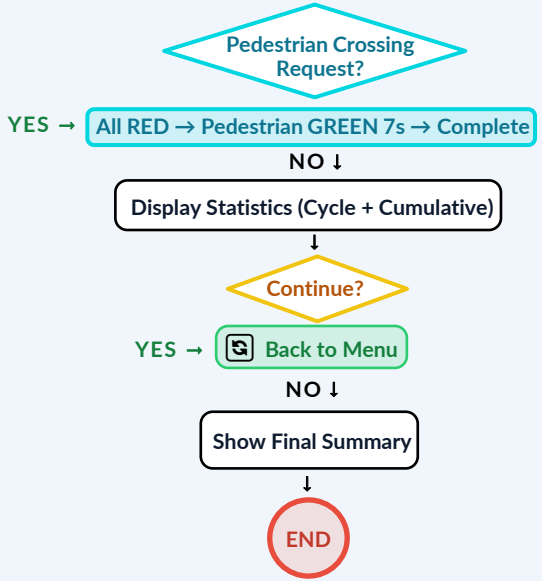
STEP 07 **PEDESTRIAN + STATISTICS** — pedestrian crossing check, then display stats

Program Flowchart

— MAIN LOOP



— POST-CYCLE



Implementation Overview

HEADER FILES

`stdio.h`	Input/output operations (printf, scanf)
`stdlib.h`	qsort(), system()
`windows.h`	Console color (SetConsoleTextAttribute), Sleep()
`string.h`	strcpy() for road name initialization

DATA STRUCTURE

```c
// Structure to hold road information
typedef struct {
    char name[10]; // Direction name
    int vehicles; // Vehicle count
    int direction;  // 1=N, 2=S, 3=E, 4=W
    int greenTime; // Computed adaptive green time
} Road;
```

This single array holds all 4 directions (North, South, East, West) — each as one `Road` struct.

Key Functional Modules

Function	Role
`main()`	System entry, main loop, cycle orchestration
`calculateGreenTime()`	Adaptive timing for normal traffic
`calculateEmergencyGreenTime()`	Adaptive timing for emergency vehicles
`runPriorityBasedTrafficCycle()`	Sorts & runs all 4 roads by priority
`processTrafficCycle()`	Runs GREEN → YELLOW → RED for one road
`handleEmergency()`	Manages emergency override (multi-direction)
`processSingleEmergency()`	Emergency cycle for single direction
`handlePedestrian()`	Pedestrian crossing with 7s GREEN

Key Functional Modules

Function	Role
`displaySignalStatus()`	Shows colored signal status for each road
`displayStatistics()`	Cycle & cumulative statistics display
`getValidInput()`	Integer input validation (range-checked)
`getYesNoInput()`	Yes/No (0 or 1) input validation
`getVehicleCount()`	Non-negative vehicle count input
`countdown()`	Real-time second-by-second countdown
`setColor()`	Console text color control
`compareRoads()`	qsort comparator (descending order)

Emergency Vehicle Handling

4-WAY SIGNAL STATE — EMERGENCY ON NORTH

NORTH ✦
GREEN EMERGENCY ACTIVE
8 / 12 / 15 sec

SOUTH
RED VEHICLES HELD
Until emergency clears

EAST
RED VEHICLES HELD
Until emergency clears

WEST
RED VEHICLES HELD
Until emergency clears

KEY SAFETY RULES

- No duplicate directions allowed per cycle
- Sequential processing for multi-direction emergencies
- Total emergency vehicles summarized per cycle

EMERGENCY PHASE FLOW

⚠ Detect Emergency ▸ ▤ Select Direction + Count ▸ ✦ Compute Emergency Green ▸ ✦ Others RED · Direction GREEN ▸ ✓ YELLOW 3s → RED · Cleared ✓

Pedestrian Crossing Module

SIGNAL STATE DURING PEDESTRIAN PHASE

North	• RED
South	• RED
East	• RED
West	• RED
PEDESTRIAN	• GREEN — 7S



PROCESS

1. All 4 vehicle signals → RED
2. Pedestrian signal → GREEN
3. Countdown: 7 seconds (fixed)
4. Pedestrian crossing marked complete
5. System returns to next cycle

SAFETY FEATURE

No vehicle direction has GREEN while pedestrians cross — ensuring zero conflict between vehicles and pedestrians during the crossing phase.

Result and Analysis

ALGORITHM COMPLEXITY

Component	Time Complexity	Space Complexity
qsort() for 4 roads	O(n log n) = O(1)	O(1)
Signal cycle (4 roads)	O(n)	O(n)
Input validation loop	O(k)	O(1)
Emergency handling	O(d)	O(d)
Overall system (per cycle)	O(c × n) → O(1)	O(n)

LIMITATIONS

Windows-only
Uses `windows.h, sleep(), getch()`

Simulated Timing
Not connected to real hardware

Fixed 4-direction
Intersection shape is hardcoded

Sequential only
No concurrent/parallel signal processing

Conclusion and Future Improvements

WHAT WE ACHIEVED

- ✓ Adaptive timing reduces unnecessary wait times
- ✓ Emergency vehicles handled with configurable priority
- ✓ Pedestrians safely managed with full vehicle RED phase
- ✓ Robust error handling makes system fool-proof
- ✓ Cumulative statistics for traffic analysis

FUTURE IMPROVEMENTS

- ❄ Cross-platform support
- ▦ GUI Interface
- ✦ Sensor Integration
- ◯ Machine Learning
- ✦ Multi-intersection
- ▦ Database Logging
- ☎ Concurrent Signals
- ◔ Real-time Clock

Thanks!



